# Spectra 提示词工程技术亮点说明

Spectra 项目组

2026-03-28

## 目录

**1 研究背景与技术目标** **2**

**2 方法来源与设计依据** **2**

**3 Spectra 提示词工程的核心优势** **3**

3.1 从自由表达提升为结构化合同 . . . . . 3

3.2 从单点提示升级为系统级语义控制层 . . . . . 3

3.3 从结果导向转向规划导向 . . . . . 3

3.4 从经验调参转向可验证优化 . . . . . 3

**4 优化设计原则** **4**

4.1 结构化上下文优先于措辞堆叠 . . . . . 4

4.2 按任务拆模板，而非复用一套大提示词 . . . . . 4

4.3 先规划，再生成 . . . . . 4

4.4 提示词变更必须进入验证链路 . . . . . 4

**5 模块级技术实现** **4**

5.1 共享 RAG / History / Session 上下文层 . . . . . 4

5.2 Chat 回复 Prompt . . . . . 5

5.3 Courseware 生成与修改 Prompt . . . . . 5

5.4 Outline 生成 Prompt . . . . . 5

5.5 Intent 与 Modify-Intent 识别 . . . . . 5

**6 验证方式与结果** **6**

**7 技术效果与系统价值** **6**

**8 结论** **6**

# 1 研究背景与技术目标

Spectra 面向的是递归式教学内容生成场景，而非单次文本补全或单轮课件排版任务。系统需要在会话问答、资料检索、大纲规划、课件生成、课件修改与结果沉淀之间维持统一语义，因此，提示词工程在 Spectra 中承担的并不是“润色提示语”这样局部的职责，而是跨模块的任务建模、上下文组织与输出约束设计。

从技术角度看，Spectra 的提示词工程目标可以概括为三点。其一，将原本依赖经验堆叠的自由文本提示，重构为具有清晰任务边界的显式合同。其二，使不同模块共享稳定的语义框架，降低长上下文场景下的漂移和歧义。其三，使提示词设计具备可验证性与可演化性，从而为系统级生成质量优化提供持续支撑。

**插图占位符：Spectra 提示词工程作用范围总览**

*建议：建议绘制一张覆盖 Chat、RAG、Outline、Courseware、Intent、Quality Eval 的链路图，并突出 Prompt 作为中间控制层而非单点文案。*

图 1: Spectra 提示词工程作用范围总览

# 2 方法来源与设计依据

本轮提示词工程优化主要参考了若干具有代表性的外部资料。OpenAI Prompt Engineering Guide 与 OpenAI Best Practices 系统总结了结构化指令、上下文组织和输出约束等通用策略； Anthropic Prompt Engineering 与 Prompt Templates and Variables 强调了任务分解、模板化和可复用变量接口；DSPy 与 promptfoo 代表了提示词优化从经验调参走向可编译、可对比和可回归验证的工程方法；LlamaIndex Prompt Engineering for RAG 提供了检索增强场景下上下文注入与回答约束的成熟模式。

在面向教学内容生成和结构化视觉生成的参考中，TeachMaster 展示了从内容规划到生成与验证的分阶段设计思路；Awaking the Slides 强调了 slide 驱动的交互式教学组织方式；Paper2Poster 体现了 top-down planning 与 layout-aware generation 的价值；From Words to Structured Visuals 则展示了如何从自然语言过渡到结构化视觉表示。这些项目与论文共同构成了 Spectra 提示词工程设计的重要方法来源。

这些资料并未被机械复制到 Spectra 中，而是被提炼为四类设计原则：其一，使用“任务目标 + 输入材料 + 约束 + 输出格式” 的显式 prompt contract；其二，按任务拆分模板，避免对话、大纲、课件和意图识别共用一套自由表达；其三，优先生成中间结构或规划，再生成最终内容；其四，将提示词设计纳入样本、测试和回归链路。

# 3 Spectra 提示词工程的核心优势

## 3.1 从自由表达提升为结构化合同

传统的生成系统往往把提示词理解为“给模型更多描述”。Spectra 的做法并非简单增加指令长度，而是把提示词重构为结构化合同，使模型清楚知道当前任务是什么、可使用哪些材料、必须满足哪些约束、最终应输出何种结构。这种方式的价值在于，它减少了任务理解的不确定性，并显著提升了多模块协作时的语义一致性。

**插图占位符：结构化 Prompt Contract 示意图**

建议：建议绘制一张四段式 *prompt contract* 示意图，展示“任务目标、输入材料、约束条件、输出格式”四个层次之间的关系。

图 2: 结构化 Prompt Contract 示意图

## 3.2 从单点提示升级为系统级语义控制层

Spectra 的提示词工程并不局限于某一个生成入口，而是同时覆盖会话问答、检索增强、大纲规划、课件生成、课件修改和意图识别等多个关键模块。这意味着提示词不再是局部写法，而是系统行为的一部分。其优势在于：不同模块可以围绕统一语义框架运作，从而降低不同子链路各说各话、局部修补彼此打架的问题。

## 3.3 从结果导向转向规划导向

受 TeachMaster、Paper2Poster 等工作启发，Spectra 并不鼓励模型直接从长上下文跳到最终输出，而是强调先规划、后生成的过程控制。在大纲生成中，模型需要先建立教学目标与章节推进；在课件生成中，模型需要先完成页面目标、材料吸收与内容布局的规划，再组织最终表达。这一设计使生成结果更具可教性和可解释性，而不是停留在表层流畅。

## 3.4 从经验调参转向可验证优化

Spectra 的提示词工程不是依赖个体经验反复试错，而是通过样本、回归测试和基线核对来约束。这意味着提示词修改可以被重复验证，可以解释“为什么有效”，也可以在未来被系统化扩展为更成熟的 prompt evaluation 工作流。这种工程纪律本身就是该系统技术亮点的一部分。

# 4 优化设计原则

## 4.1 结构化上下文优先于措辞堆叠

提示词质量问题在 Spectra 中往往并非来自模型“不懂要求”，而是来自上下文组织失控。因此，本轮优化首先收束共享上下文层，将 RAG、历史消息、会话状态和任务目标组织为稳定分块，而不是继续叠加自由文本描述。这样做的直接收益是减少模型在上下文边界处的语义漂移，使引用、任务意图和输出约束具有更明确的位置。

## 4.2 按任务拆模板，而非复用一套大提示词

对话回复、大纲生成、课件生成、课件修改、意图识别和修改意图解析属于不同性质任务。本轮调整将这些任务分别建模：对话侧强调任务推进与资料不足时的约束；大纲侧强调教学目标、章节推进与页数分配；课件生成侧强调规划规则与输出契约；意图识别侧强调边界清晰与类别互斥。其核心思想与 Anthropic 和 OpenAI 的结构化模板建议一致，即不同任务必须拥有不同的 success criteria。

## 4.3 先规划，再生成

受 TeachMaster 与 Paper2Poster 等工作的启发，本轮优化进一步强化了 Spectra 的“先规划、后生成”思路。Outline prompt 被要求明确章节教学任务和结构推进，课件生成 prompt 则在进入最终内容表达前先固定页面目标、材料利用和输出块契约。这一策略降低了直接从长上下文跳到最终课件文本时的随机性。

## 4.4 提示词变更必须进入验证链路

本轮所有重要提示词工程改动都伴随了测试更新。其方法不是让评测系统替代业务逻辑，而是确保 prompt contract 变化能通过已有测试、静态检查和样本验证被观察到。该原则与 promptfoo、simple-evals 和 DSPy 中强调的“prompt optimization must be measurable”是一致的。

# 5 模块级技术实现

## 5.1 共享 RAG / History / Session 上下文层

共享上下文层的主要问题是历史、检索结果和任务说明在提示词里混杂，容易造成模型错误理解信息来源与优先级。本轮调整将其重构为更稳定的语义分块，使模型能区分当前任务目标、会话历史、检索材料以及引用约束。

这一改动的收益包括：第一，引用要求不再淹没在长上下文中；第二，资料不足时的回答边界更明确；第三，后续各模块可以复用统一上下文组织方式，而不是各自拼装上下文。

## 5.2 Chat 回复 Prompt

对话 prompt 的优化重点是把“生成一个好回答”改写为“在当前任务上下文中推进问题解决”。新的 contract 强调三点：一是优先推进用户当前问题，而不是生成风格化冗余文本；二是在资料不足时显式保持保守，不伪造确定性结论；三是需要追问时应有明确触发条件，而不是默认转入追问。

这使得对话模块在长会话和资料密集场景下的行为更可控，也更符合 Spectra 的“会话是工作展开而不是闲聊容器”的系统定位。

## 5.3 Courseware 生成与修改 Prompt

课件生成 prompt 的主要问题是任务、输入、规划规则和输出格式长期混在同一层级，导致模型有时更像在“自由写作”，而不是按约束产出结构化课件内容。本轮优化将其重写为显式 contract：先明确任务目标和输入材料，再给出规划约束，最后定义输出块格式。

对修改 prompt，则强调局部修改优先和教学结构保持，即模型在执行修改时不应无约束重写整个课件，而应尽量维持原有教学推进和输出块结构。这样既提高了修改质量，也增强了修改行为的可预期性。

## 5.4 Outline 生成 Prompt

大纲 prompt 的优化重点在于把“页面列表生成”升级为“教学结构规划”。新的设计强调：先根据目标和材料确定章节教学任务，再做内容推进和页数分配，最后输出更稳定的大纲结构。

这与此前已落地的大纲去重 heuristics 相互补充：heuristics 负责抑制明显重复和结构失衡， prompt 则负责在更高层面提升结构推进和教学完整性。

## 5.5 Intent 与 Modify-Intent 识别

意图识别属于 Spectra 中最容易被忽视、但对交互稳定性影响极大的提示词模块。此前意图分类容易在 `describe_requirement`、`ask_question`、`modify`、 `confirm` 等类别间产生边界模糊。本轮优化的主要目标是把分类条件和互斥关系写清，并保留关键词 fallback 作为保守兜底。

对修改意图解析，则进一步强调结构化输出合同，使后续链路可以依赖更稳定的意图解析结果，而不是从自由文本中再次猜测。

**插图占位符：Spectra 提示词工程模块分布图**

建议：建议用模块图展示共享上下文层、*Chat*、*Outline*、*Courseware*、*Intent* 五类 *Prompt* 如何分别承担不同职责，并共同构成系统级控制层。

图 3: Spectra 提示词工程模块分布图

# 6 验证方式与结果

本轮提示词工程优化并未绕开工程验证流程，而是通过现有测试与静态检查进行约束。与提示词相关的主要验证覆盖包括共享上下文、对话 prompt、课件生成与修改、大纲生成、意图识别和修改意图解析。回归测试结果显示，相关测试共 89 项全部通过。

除回归测试外，本轮还对每个功能点执行了格式化和静态检查，以防止提示词相关改动通过无关代码质量问题污染分支状态。这里的重点不是把提示词工作伪装成“纯文档修改”，而是将其纳入与正常后端变更一致的工程纪律。

# 7 技术效果与系统价值

当前优化已经在三个层面提升了系统能力。第一，共享上下文与任务边界更清晰，降低了会话、检索和任务说明混杂带来的语义漂移。第二，课件与大纲 prompt 的规划性增强，使输出更符合教学推进逻辑，而不是停留在表层流畅。第三，意图识别与修改解析更稳定，增强了交互式生成流程中的可控性和一致性。

从更高层面看，Spectra 的提示词工程优势不只是“生成结果更稳”，而是使教学内容生成系统具备了更强的结构性、可解释性与可演化性。它将提示词从经验性的生成技巧，提升为能够支撑系统长期扩展的工程控制机制。

# 8 结论

Spectra 的提示词工程不应被简单理解为“把文案写得更像提示词工程”，而应被理解为一次围绕系统语义、任务规划和工程验证的结构化升级。本轮工作已经将主要 prompt 入口从自由表达进一步推进到可解释、可验证、可回归的 contract 形态，并系统性提升了对话、检索上下文、课件生成、大纲生成与意图识别的稳定性。

从工程视角看，这一轮工作的最大价值不在于单点 prompt 性能提升，而在于为 Spectra 建立了一套可持续、可扩展、可验证的提示词工程基础。这一基础使系统能够在保持语义一致的同时，继续向更高质量的教学内容生成与更强的多产物协同能力演进。